

TCSS 487 Cryptography

Practical Project - Cryptographic Library and App - Part 1

Students: Rudolf Arakelyan and Linda Miao

UW Tacoma - Spring 2026

Instructor: Prof. Paulo L. Barreto

1. Overview

We implemented the Part 1 symmetric cryptography library and command-line application in Java. The project provides SHA-3 hashing, SHAKE extendable-output functions, SHAKE-based authentication tags, and symmetric file encryption/decryption at the required 128-bit security level. All cryptographic SHA-3/SHAKE functionality is implemented in our own SHA3SHAKE.java class rather than by Java built-in hash classes or third-party libraries.

The application is command-line based. All file names, output file names, passphrases, direct text inputs, and requested output lengths are supplied through the String[] args argument to Main.main(), matching the grading requirements.

2. Submitted Source Files

File	Purpose
Main.java	Required application entry point. Parses command-line arguments and exposes hashing, MAC, encryption, and decryption services.
SHA3SHAKE.java	Required SHA-3/SHAKE class. Implements the Keccak sponge, SHA-3-224/256/384/512, SHAKE-128/256, and the exact assignment interface.
Test.java	Local self-test helper with no main method. It is included as source only and does not replace Main.java as the application entry point.
TCSS487_Project1_Report.pdf	Project report with usage instructions, solution description, validation notes, known limitations, and attribution.

3. SHA3SHAKE.java Implementation

SHA3SHAKE.java implements a plain, non-duplexing Keccak sponge with a 1600-bit state stored as 25 Java long values. Bytes are absorbed and squeezed in little-endian order within each 64-bit lane, as required by Keccak. The permutation is the 24-round Keccak-f[1600] function with theta, rho, pi, chi, and iota steps.

The class follows the interface from the project specification. Calling init(int suffix) resets the state. absorb() is allowed only after init() and before the first squeeze() or digest(). Repeated squeeze() calls return consecutive output chunks. Repeated digest() calls return the same fixed SHA-3 digest after the first digest call. digest() and squeeze() are mutually exclusive until another init() call resets the sponge.

Function	Rate (bytes)	Capacity (bits)	Domain byte
SHAKE-128	168	256	0x1F
SHAKE-256	136	512	0x1F
SHA3-224	144	448	0x06
SHA3-256	136	512	0x06

Function	Rate (bytes)	Capacity (bits)	Domain byte
SHA3-384	104	768	0x06
SHA3-512	72	1024	0x06

Because the suffix value 256 is used by both SHA3-256 and SHAKE-256, the implementation chooses the correct domain byte based on the operation: `digest()` uses the SHA-3 domain byte 0x06, while `squeeze()` uses the SHAKE domain byte 0x1F.

4. Main.java Application Usage

4.1 Compile

From the project directory, compile with:

```
javac SHA3SHAKE.java Main.java Test.java
```

The generated .class files are build outputs only and must not be included in the submission ZIP.

4.2 Hash a File - Required and Bonus

This service reads a user-specified file as bytes and prints SHA3-224, SHA3-256, SHA3-384, and SHA3-512. SHA3-256 and SHA3-512 satisfy the required hash item. SHA3-224 and SHA3-384 satisfy the hash bonus.

```
java Main hash <file>
```

4.3 MAC a File - Required

This service computes a SHAKE authentication tag for a file under a user-specified passphrase. The program absorbs the passphrase bytes first, then the file bytes, and finally squeezes the requested number of bytes.

```
java Main mac <128|256> <file> <passphrase> <outputLengthBytes>
java Main mac <file> <passphrase> <outputLengthBytes>
```

The shorter form defaults to SHAKE-128 for backward compatibility.

4.4 MAC Direct Text - Bonus

This bonus service uses the same MAC construction as the file MAC service, but the data is taken directly from a command-line text argument instead of from a file.

```
java Main mactext <128|256> "<text>" <passphrase> <outputLengthBytes>
java Main mactext "<text>" <passphrase> <outputLengthBytes>
```

The shorter form defaults to SHAKE-128. Text or passphrases with spaces should be quoted in the shell.

4.5 Encrypt a File - Required and Bonus

Encryption derives a 128-bit symmetric key by computing SHAKE-128(passphrase, 128 bits). It samples a random 16-byte nonce with Java `SecureRandom`. SHAKE-128 is used as a stream cipher by absorbing nonce || key and squeezing a keystream equal to the plaintext length. The ciphertext is plaintext XOR keystream.

For the bonus MAC, encryption computes SHA3-256(key || ciphertext) and appends the 32-byte tag to the cryptogram.

```
java Main encrypt <inputFile> <passphrase> <outputFile>
java Main encrypt <inputFile> <passphrase>
```

The shorter form writes to `<inputFile>.enc`.

4.6 Decrypt a File - Required and Bonus

Decryption parses the nonce, ciphertext, and tag from the cryptogram. It derives the same key from the passphrase, recomputes SHA3-256(key || ciphertext), and compares it to the stored tag before decrypting. If the tag does not match, the program rejects the cryptogram and does not write plaintext.

```
java Main decrypt <inputFile> <passphrase> <outputFile>
java Main decrypt <inputFile> <passphrase>
```

The shorter form writes to <inputFile>.dec, or replaces a trailing .enc extension with .dec.

5. Cryptogram Format

Every encrypted output file produced by Main.encrypt() uses this byte layout:

```
[16-byte nonce] || [ciphertext bytes] || [32-byte SHA3-256 MAC]
```

The MAC is computed over key || ciphertext, not over the plaintext. This lets the decrypt command verify integrity and reject wrong passphrases or tampered ciphertext before writing a decrypted output file.

6. Validation

We validated the SHA-3 and SHAKE implementation against known FIPS 202/NIST test vectors and tested the application workflows end to end.

Test	Expected / Observed result	Status
SHA3-224("")	6b4e03423667dbb73b6e15454f0eb1abd4597f9a1b078e3f5b5a6bc7	PASS
SHA3-256("")	a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a	PASS
SHA3-384("")	0c63a75b845e4f7d01107d852e4c2485... length 48 bytes	PASS
SHA3-512("")	a69f73cca23a9ac5c8b567dc185a756e... length 64 bytes	PASS
SHA3-256 of 0xA3 repeated 200 times	79f38adec5c20307a98ef76e8324afbdf46cfd81b22e3973c65fa1bd9de31787	PASS
SHAKE-128("", 256 bits)	7f9c2ba4e88f827d616045507605853ed73b8093f6efbc88eb1a6eacfa66ef26	PASS
SHAKE-256("", 512 bits)	46b9dd2b0ba88d13233b3feb743eeb24... length 64 bytes	PASS
Encrypt/decrypt round trip	Recovered file exactly matched original file by cmp	PASS
Wrong passphrase decrypt	MAC verification failed and no plaintext output file was written	PASS

7. Known Bugs and Limitations

- No known functional bugs at submission time.
- The application reads each input file fully into memory, so extremely large files may require substantial RAM.
- The mactext service receives the message from the shell command line. Long messages or messages containing special shell characters may need careful quoting.
- The project is an educational implementation and does not attempt to provide a production file format beyond the project-specified nonce, ciphertext, and MAC layout.

8. Attribution and Academic Integrity

The SHA3SHAKE.java implementation was written for this project. Its structure was inspired by Markku-Juhani Saarinen's tiny_sha3 reference implementation in C:

```
https://github.com/mjosaarinen/tiny\_sha3/blob/master/sha3.c
```

No code was copied directly from that source. The C implementation was used as a structural reference, as permitted by the project specification. The Keccak round constants and rotation offsets are the FIPS 202 constants. The program does not use Java SHA-3, Java SHAKE, MessageDigest, or any third-party cryptographic implementation. Java SecureRandom is used only as the external randomness source for the required 128-bit encryption nonce.

9. Submission Contents

The submission ZIP should contain only the Java source files and this PDF report. It should not contain .class, .jar, .exe, .enc, .dec, .git, DOCX, or assignment handout files.

```
Main.java  
SHA3SHAKE.java  
Test.java  
TCSS487_Project1_Report.pdf
```